Supplementary Material: CL-TAV Contracts, Wrappers and Decision Tables

Author Name (affiliation pending)

Abstract

This supplement is part of the delivered manuscript package. It typesets ALG-CLTAV-02/03/04 wrappers, ALG-CLTAV-05/06/07 cores, typed records and the A/P stop tables. It does not change ALG-CLTAV-01 and does not report experiments. Supplement algorithm numbers are S1, S2, ...; they are not the main-text Algorithm 1.

1 Scope and numbering

Main-text Algorithm 1 is ALG-CLTAV-01. Supplement Algorithms S1–S6 are ALG-CLTAV-02, 03, 04, 05, 06 and 07. S9 in the main text calls CommitCompatibleUpdate (S6 / ALG-CLTAV-07), not a bare `IF-HIST-UPDATE` invocation.

2 S0–S10 duty map

Step	Call	Duty / exception
S0	Initialize	Γ, η, H, U; distinct k, charges, retry
S1	`IF-RESOURCE-STOP` with `IF-EQUIV`	already decided Admit A2–A5 or P-stop
S2	PredictCurrent	ALG-CLTAV-02; TEST-scoped GAP
S3	SelectAndAdmit	ALG-CLTAV-05; freeze ξ including U
S4	ChargeOnce	one charge; retry cap $\neq$ budget
S5	`IF-EXECUTE-RECORD`	raw record and effect class
S6	InterpretOutcome	ALG-CLTAV-03 + timing ALG-CLTAV-06
S7–S8	ResolveOutcome	ALG-CLTAV-04; adopt returned contexts
S9	CommitCompatibleUpdate	version/intersection; SPEC-ERROR; summary commit
S10	`IF-RESOURCE-STOP` with `IF-EQUIV`	P-stop or continue to S1

3 Typed records

Record	Fields
PredictionResult	status OK|GAP; reason; classesByTest (OK); historyVersionUsed; uncertaintyRef
Decision	kind ACTION|EXIT|SPEC-ERROR; actionKind; actionId; classesUsed; reason
SelectSnapshot ξ	actionId; actionKind; qUsedAtSelect; historyVersion; classesUsedAtSelect; uncertaintyRef
ExecutionResult	attemptId; record; effect; correlationId
Outcome z	validity; effect; kind; actionId; I_z or NONE; prepResultEvaluated; targetConfirmed; summaryConfirmed; prepErr; declaredTarget; evidence; postSummary; timing
Resolution δ	control COMMIT|RETRY|STOP; Γ'; η'; outcome; reason

4 Admit and post-update stops

A1 selects one affordable eligible action from the exclusive TEST, Prep or Recover alternatives; distinguishing value is required only for TEST. A2 is KNOWN with a distinguishing TEST or eligible Prep, but no such action affordable (Stop-Budget). A3 is KNOWN with no distinguishing TEST and no eligible Prep (Stop-NoDistinguisher). A4 is UNKNOWN with no eligible Recover (Stop-Error). A5 is UNKNOWN with Recover eligible but unaffordable (Stop-Budget). A TEST prediction GAP is handled independently: a KNOWN GAP without eligible Prep is SPEC-ERROR, not A3; retry-cap remains an independent Stop-Error condition. P1 Stop-Empty; P2 Stop-Singleton; P3 Stop-645; P4 Stop-Equivalent; P5 Stop-Budget after update. Stop-NoDistinguisher is exclusive with those P-stops when no remaining distinguisher exists and no P-stop already fired. Order is Empty, Singleton, 645, Equivalent, Budget. Singleton h_{normal} is not protocol PASS.

5 Short arguments

Monotonicity, retention (R1)–(R5), one-step minimax and resource termination are stated in the main text. The algebraic details are the accepted method §3.9.1 arguments: update is intersection; `ERROR` does not exclude; minimax is a least element of a finite selectable set; billed executions are bounded by the selected mode. They are not global optimality and not independent mathematical approval.

6 Experiment-design accounting contract

The candidate design reuses four arms, three families and eight registered scenario identifiers. One unique `attemptId` is the all-ledger unit; `runId` exists only after admission. Truth joins arm output only at evaluation. Detection answered, localization decision, true-hypothesis retention, singleton correctness, abstention, error, equivalence, unresolved fault and budget exit remain distinct, potentially overlapping labels. Every result cell names its finite population, missing/censoring rule and metric-specific empty-denominator behavior. With registered attempts but no answers, answered coverage is (0/N) while answered-conditional error is N/A. Recovery detail is folded into preparation exactly once in the accepted four-component cost projection. Pre-run review/setup effort is reported separately from issued-action cost. An `ERROR` preserves the preceding hypothesis set; only a valid empty compatibility update produces `Stop-Empty`. A registered pair requires the same base scene/seed, reciprocal arms, distinct reset evidence and parent metric eligibility. The closed-interval T5 rule assigns containment PASS, disjointness FAIL and partial overlap INCONCLUSIVE. Confirmatory execution is blocked until the IUT/reset, truth/injection, equivalence, mathematical-review, sample/repeat, resource-budget and clock/error budget decisions are frozen. The machine-checked synthetic ledger is an accounting witness, not evidence of method performance.

7 AI disclosure fields

System: PENDING-USER (AUTH-AI). Scope (text/figures/code): PENDING-USER. Author verification: required; not completed by this sentence. IEEE AESS guidance is rechecked at submission.

8 Wrappers and core mechanisms

Algorithm S1: PredictCurrent: current-observation prediction wrapper (ALG-CLTAV-02, S2)

Require: SessionContext Γ; HistoryHandle η; current H; uncertainty reference U; action library L

Ensure: PredictionResult: status OK with nonempty classesByTest, or status GAP with a reason; the session's own η is read, not rebuilt

$raw \leftarrow$ `IF-PRED-OBS`(Γ, η, H, U, L);

`// branch on the backend tag BEFORE reading or projecting classes; a GAP result carries no classesByTest`

if raw.status = *GAP* **then**

  return PredictionResult(status = `GAP`, reason = raw.reason);

  `// the gap stays TEST-scoped`

$classes \leftarrow$ ProjectOntoCurrentH(raw.classes, H);

`// a declared class set that projects to no nonempty class is the same named gap, not OK and not` $score = 0$

if *classes has no nonempty member* **then**

  return PredictionResult(status = `GAP`, reason = no currently valid class);

return PredictionResult(status = `OK`, classesByTest = *classes*, historyVersionUsed = η.version, uncertaintyRef = U);

Algorithm S2: InterpretOutcome: execution and observation coordination (ALG-CLTAV-03, S6)

Input: ExecutionResult er; SelectSnapshot ξ; Γ, η; uncertainty reference U
Output: total Outcome z, including effect, validity, I_z, and confirmation fields

$z \leftarrow$ NewOutcome(); z.effect $\leftarrow$ er.effect; z.kind $\leftarrow$ ξ.actionKind; z.actionId $\leftarrow$ ξ.actionId;
if er.effect $\in$ {*CONFIRMED-NOT-SENT*, *UNKNOWN-EFFECT*} **then**
    return z;
$(I_z, e, s, p, o, m) \leftarrow$ `IF-OBS-INTERPRET`$(\Gamma, er.\text{record}, \text{clocks}, \varepsilon)$;
if o *is invalid* **then**
    set z.validity $\leftarrow$ ERROR and z.effect $\leftarrow$ `UNKNOWN-EFFECT`; **return** z;
timing $\leftarrow$ InterpretTimedObservation$(o, m, \xi, N_r, D_r, \text{Match}_r)$;
if timing.verdict = ERROR **then**
    set z.validity $\leftarrow$ ERROR and z.effect $\leftarrow$ `UNKNOWN-EFFECT`; **return** z;
set z.validity $\leftarrow$ timing.verdict and $z.I_z \leftarrow I_z$;
if ξ.actionKind $\in$ {PREP, RECOVER} **then**
    `// IF-PREP-RECOVER returns values only; it does not write Γ or η`
    z.prepResultEvaluated $\leftarrow$ **true**;
    $(z.\text{targetConfirmed}, s, z.\text{prepErr}, z.\text{declaredTarget}, z.\text{evidence}, p) \leftarrow$ `IF-PREP-RECOVER`$(\Gamma, \eta, \xi.\text{actionId}, er.\text{record})$
set z.summaryConfirmed $\leftarrow$ s and z.postSummary $\leftarrow$ p;
if $e =$ *UNKNOWN-EFFECT*, z.prepErr, *an unconfirmed Prep summary, an unconfirmed Recover target, or* $e =$ *NONE with no confirmed summary* **then**
    set z.effect $\leftarrow$ `UNKNOWN-EFFECT`;
else
    set z.effect $\leftarrow$ e;
`// all Γ/η writes stay in ALG-CLTAV-04 and S9`
return z;

Algorithm S3: ResolveOutcome: ERROR, retry and confirmed recovery (ALG-CLTAV-04, S7–S8)

Require: Outcome z (effect, kind, validity, confirmation); SelectSnapshot ξ; Γ; η
Ensure: Resolution $\delta = (\text{control}, \Gamma', \eta', \text{outcome}, \text{reason})$: Γ' initialized from Γ

$\Gamma' \leftarrow \Gamma$; $\eta' \leftarrow \eta$;
`// copy/persist semantics: the returned contexts are total, never undefined`
if z.validity = ERROR *or* z.effect $\in \{$*CONFIRMED-NOT-SENT*, *UNKNOWN-EFFECT*$\}$ *or* z.prepErr *or* (*z*.kind *is Prep and* *z*.prepResultEvaluated *and* *z*.summaryConfirmed *is false) or* (*z*.kind *is Recover and* *z*.prepResultEvaluated *and* *z*.targetConfirmed *is false)* **then**
  `// S7 classify by kind and evaluated confirmation; do not fold CONFIRMED-NOT-SENT into UNKNOWN`
  do not exclude H;
  if z.validity = ERROR *or* z.prepErr *or* z.effect = *UNKNOWN-EFFECT* *or* (*z*.kind *is Prep and* *z*.prepResultEvaluated *and* *z*.summaryConfirmed *is false)* **then**
    mark η' conservative-unknown; Γ'.qStatus $\leftarrow$ `UNKNOWN`; clear Γ'.currentSummary;
  else if z.effect = *CONFIRMED-NOT-SENT* **then**
    leave Γ'.qStatus and compatible history unchanged;
  else
    Γ'.qStatus stays UNKNOWN; clear Γ'.currentSummary; mark η' conservative-unknown;
    `// unconfirmed Recover target`
  Γ'.retryCount $\leftarrow \Gamma$.retryCount + 1;
  if Γ'.retryCount $\geq \Gamma'$.retryCap **then return** Resolution(control = `STOP`, Γ', η', z, reason = Stop-Error);
  return Resolution(control = `RETRY`, Γ', η', z);
  `// an ERROR/unknown outcome reaches retry, never commit`
else if z.kind *is Recover* **then**
  `// S8 a confirmed Recover is a necessary condition for this branch; sole writer of KNOWN; the normalized summary returns to S9`
  Γ'.qStatus $\leftarrow$ `KNOWN`; z.postSummary $\leftarrow$ (z.declaredTarget, z.evidence);
return Resolution(control = `COMMIT`, Γ', η', z);
`// a normal TEST/Prep with a valid observation commits; the normalized Outcome carries the confirmed postSummary to S9`

Algorithm S4: SELECTANDADMIT: test selection and resource admission (ALG-CLTAV-05, S3)

Require: Γ; η; current H; action library L; typed *pred* (PredictionResult)
Ensure: a Decision: kind ACTION / EXIT / SPEC-ERROR with actionKind, actionId, classesUsed and reason

`// state eligibility is independent of affordability; under UNKNOWN only an eligible Recover qualifies, never a Prep`

eligible$(t) \leftarrow (\Gamma.\text{qStatus} = \texttt{UNKNOWN}\ ?\ (t.\text{kind} = \text{RECOVER} \wedge t.\text{eligibleUnknown})\ :\ (t.\text{kind} \neq \text{RECOVER} \wedge t.\text{enabledAt} = \Gamma.\text{currentSummary}))$;
$A \leftarrow \{t \in L : \text{eligible}(t)\}$;

`// affordability is decided by IF-SELECT-ADMIT, not assumed from eligibility`
`// a global or per-TEST projection gap withholds TEST scoring and keeps the gap identity; it does not itself block eligible Prep/Recover`

testGap $\leftarrow$ **false**;
if *pred*.status = *GAP* **then**
  $C \leftarrow \{\}$; *testGap* $\leftarrow$ **true**;
else
  $C \leftarrow \{\mathcal{C}_t\}$ with $\mathcal{C}_t \leftarrow \{H \cap C : C \in \textit{pred}.\text{classesByTest}[t],\ H \cap C \neq \varnothing\}$ for each TEST $t \in A$;
  if *a TEST* $t \in A$ *has declared classes but* $\mathcal{C}_t = \varnothing$ **then**
    $C \leftarrow \{\}$;
    testGap $\leftarrow$ **true**;

`// IF-SELECT-ADMIT is the single admission authority: affordability and A1-A5; exclusive kind; one-step minimax` $s(t) = \max_{C \in \mathcal{C}_t} |C|$ `over currently valid nonempty classes of affordable distinguishing TESTs` $D \cap F$ `only; cost/id tie-break; never` $\arg\min \varnothing$`; Prep/Recover are not observation minimax; uninformative TESTs are not A2`

$(k, S, t^\star, \textit{admit}) \leftarrow \texttt{IF-SELECT-ADMIT}(A, \Gamma.\text{currentSummary}, C, \Gamma, \eta, H)$;
if k = ADMIT **then**
  if *testGap and admit* = A3 **then return** Decision(kind = `SPEC-ERROR`, reason = *pred*.reason or per-test prediction gap);
  return Decision(kind = `EXIT`, reason = the A2–A5 admission reason);
else if k = PREP **then**
  return Decision(kind = `ACTION`, actionKind = `PREP`, actionId = $t^\star$, classesUsed = S);
else if k = RECOVER **then**
  return Decision(kind = `ACTION`, actionKind = `RECOVER`, actionId = $t^\star$, $\cdot$);
else if *testGap* **then**
  return Decision(kind = `SPEC-ERROR`, reason = *pred*.reason);
return Decision(kind = `ACTION`, actionKind = `TEST`, actionId = $t^\star$, classesUsed = $\mathcal{C}_{t^\star}$);

Algorithm S5: InterpretTimedObservation: timed observation kernel (ALG-CLTAV-06, S6 kernel)

Require: the single ownership result and the raw measurement interval from IF-OBS-INTERPRET; SelectSnapshot ξ; named obligation inputs N_r (allowed interval) and D_r (physical domain); declared match policy Match_r; declared uncertainty ε

Ensure: a TimingResult: verdict in {PASS, FAIL, INCONCLUSIVE, ERROR}; the single pairing evidence; the interval used; validity

`// resolve` N_r, D_r `and` ε `from the named obligation and instance inputs, not from free symbols`

pair $\leftarrow \text{Match}_r(\text{ownership}, \xi)$;

`// one pairing result feeds T2 and the no-response rule`

if pair *is ambiguous* **then return** TimingResult(verdict = ERROR);

if pair *is a legitimate trigger/response pair* **then**

  $M \leftarrow \text{measurementInterval} \cap D_r$;

  if $M = \varnothing$ **then return** TimingResult(verdict = ERROR);

  if $M \subseteq N_r$ **then**

    return TimingResult(verdict = PASS, evidence = pair, M);

  else if $M \cap N_r = \varnothing$ **then**

    return TimingResult(verdict = FAIL, evidence = pair, M);

  return TimingResult(verdict = INCONCLUSIVE, evidence = pair, M);

  `// open/closed endpoints preserved`

`// a missing response forms a no-response only after a still-active instance reaches its deadline; this round not seeing it is not a timeout`

if *the obligation is still active and expired* **then**

  `// use the earliest admissible elapsed horizon (a scalar), with the requirement's boundary convention`

  if N_r *is closed at* U_r **then**

    expired $\leftarrow$ lowerHorizon $> U_r$;

  else

    expired $\leftarrow$ lowerHorizon $\geq U_r$;

  if expired **then return** TimingResult(verdict = FAIL);

  return TimingResult(verdict = INCONCLUSIVE);

return TimingResult(verdict = ERROR);

`// a missing pair is not a point observation`

Algorithm S6: CommitCompatibleUpdate: compatible update and summary commit (S6 / ALG-CLTAV-07)

Input: snapshot ξ; Outcome z; contexts Γ, η, H
Output: UpdateResult $(status, \eta', H', \Gamma')$

$\Gamma' \leftarrow \Gamma$;
if ξ.historyVersion $\neq \eta$.version **then return** SPEC-ERROR with unchanged η, H, Γ';
if $z.I_z$ *is a valid compatible class* **then**
    $(\eta_c, H_c) \leftarrow$ `IF-HIST-UPDATE` with:;
        η, H, ξ.actionId, ξ.qUsedAtSelect, ξ.classesUsedAtSelect,;
        ξ.historyVersion, $z.I_z$, z.postSummary;
    if $H_c \neq H \cap z.I_z$ **then return** SPEC-ERROR (intersection mismatch);
    $\eta' \leftarrow \eta_c$; $H' \leftarrow H_c$;
else
    $(\eta', H') \leftarrow$ `IF-HIST-UPDATE` with the same snapshot fields, NONE, and z.postSummary;
    $H' \leftarrow H$;
if Γ'.qStatus *is KNOWN and* z.summaryConfirmed **then**
  Γ'.currentSummary $\leftarrow z$.postSummary;
return OK with (η', H', Γ');